

Lenguaje de Presentación
CSS - Cascading Style Sheets

Técnicas de Posicionamiento y Maquetación

Dra. B. Cristina Pelayo García-Bustelo
Departamento de Informática
Universidad de Oviedo
crispelayo@uniovi.es

Grado en Ingeniería Informática del Software

MODELO DE FORMATO VISUAL

Escuela de Ingeniería Informática
Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Modelo de formato visual

- Los elementos HTML generan visualmente una o varias cajas que CSS puede posicionar y modificar

- Con CSS se controla:
  - Tamaño
  - Posición
  - Separación
  - Apariencia

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Modelo de formato visual

- Elemento HTML de bloque (pj <p> o <section>)
  - Ocupa el ancho disponible
  - Comienza en una nueva línea
- Elemento HTML de línea (pj <a> <img>)
  - Ocupa únicamente el espacio necesario
  - Continúa en la misma línea

enlace

Un párrafo con un enlace forma una caja que ocupa el ancho existente

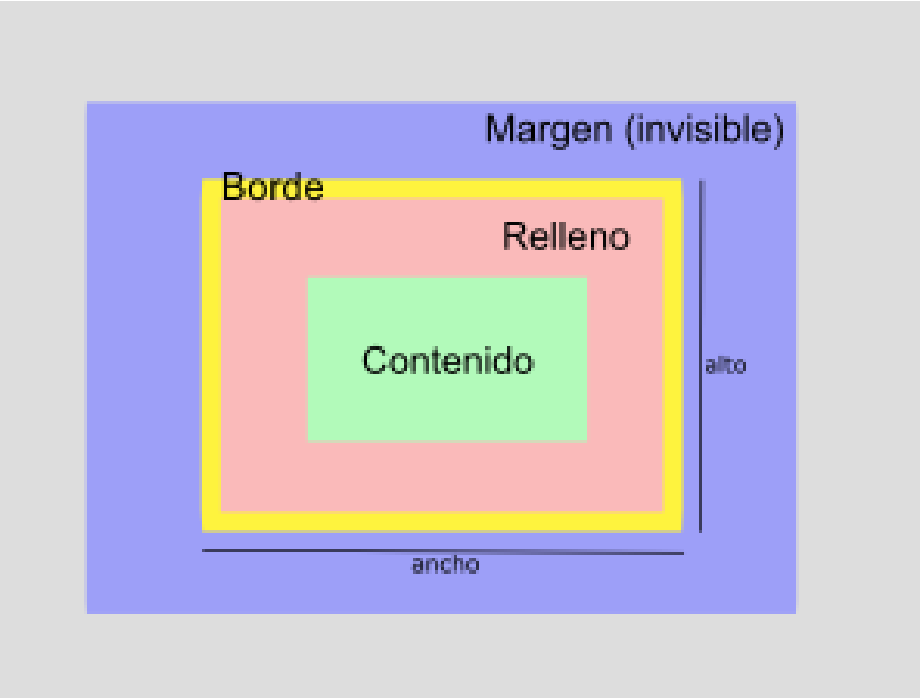
El segundo párrafo forma otra caja

Un párrafo con un enlace forma una caja que ocupa el ancho existente

El segundo párrafo forma otra caja

Modelo de caja

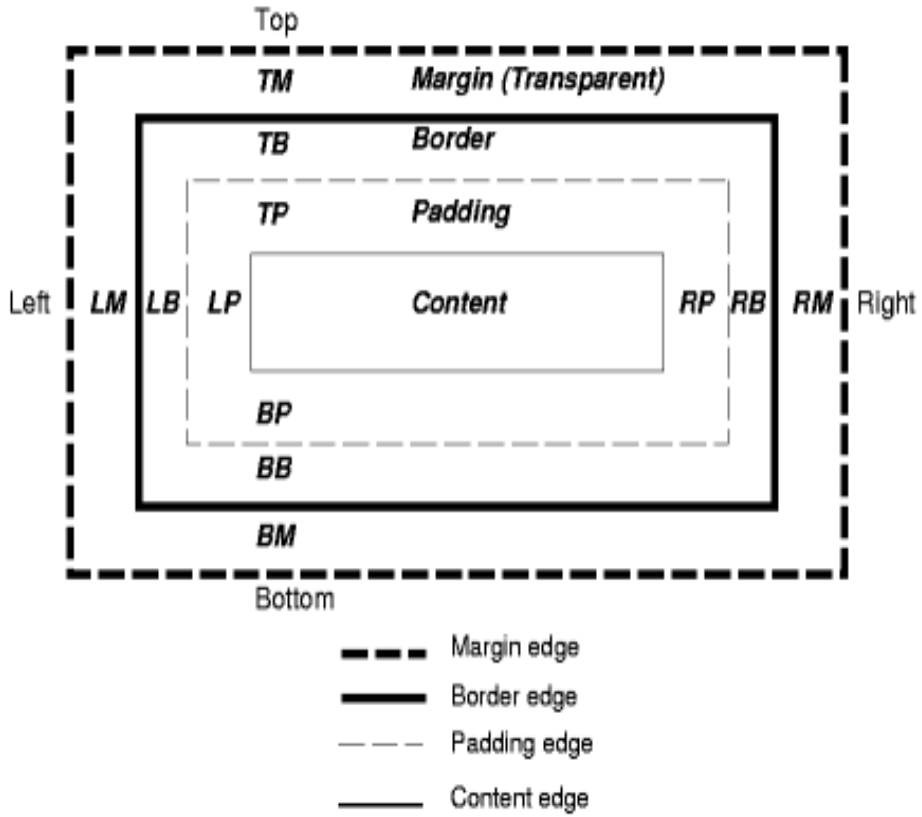
- El tamaño y posición del elemento son relativos a la caja que forma



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Modelo de caja

- Propiedades relacionadas:
  - Margin
  - Border
  - Padding
  - Width
  - Height



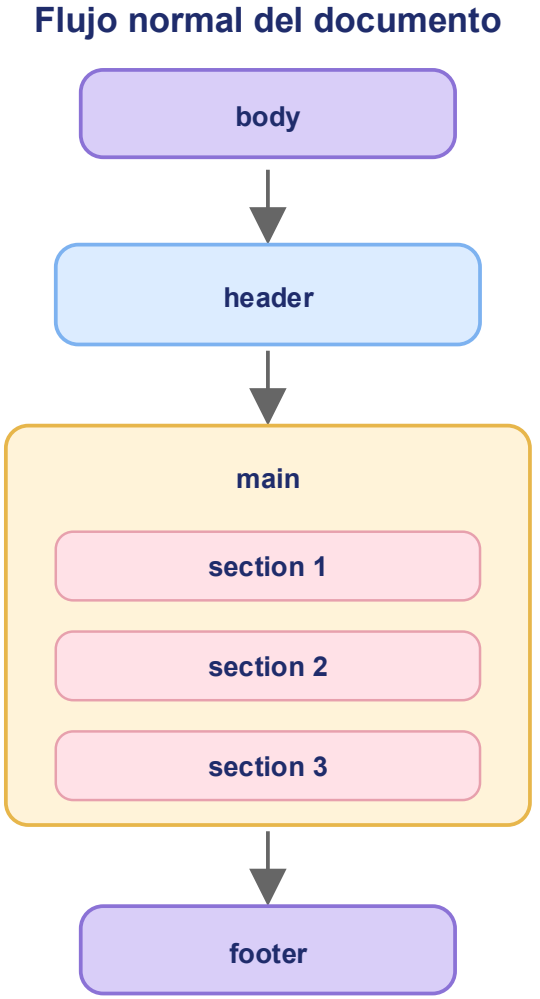
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Modelo de formato visual

Flujo normal: el flujo normal incluye formato de bloque de cajas de bloque, formatos en línea de cajas en línea, posicionamiento relativo de cajas de bloque o en línea.

- Las cajas se sitúan según su tipo siguiendo el orden de especificación en HTML,

Definir los contenidos estructurados siguiendo el flujo normal ayuda a la accesibilidad



Los elementos aparecen siguiendo el orden definido en el HTML

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Modelo de formato visual

Propiedad display

- Especifica cómo se representa un elemento y el tipo de caja que genera

```
display : inline | block | inline-block|
          ……|
        flex |
         grid | none
```

Otros valores son:

- inline: el elemento genera una o mas cajas de línea.
- block: el elemento genera una caja de bloque principal.
- inline-block: el elemento genere una caja de bloque, que fluye como una caja en línea.
- none: no genere ninguna caja en la estructura del formato

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grado en Ingeniería Informática del Software

POSICIONAMIENTO TRADICIONAL

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Flujo normal:

- Los elementos aparecen siguiendo el orden del HTML

Flotantes:

- Permiten que el contenido fluya alrededor
- La caja se presenta primero de acuerdo al flujo normal, luego se saca del flujo normal y se mueve a la izquierda o derecha tanto como sea posible

Posicionamiento absoluto:

- La caja es sacada completamente del flujo normal y se posiciona respecto a un contenedor

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional


```
position : static | relative | absolute | fixed | sticky
```

La propiedad position permite ajustar la ubicación de elementos concretos dentro de una página.

Cuando se puede utilizar

- Menús fijos (fixed)
- Cabeceras adhesivas (sticky)
- Etiquetas sobre imágenes (absolute)
- Botones flotantes

No se recomienda para

- Estructurar una página completa
- Crear columnas
- Diseños responsive complejos
- Para esto se recomienda utilizar Flexible Box Layout y Grid

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional


```
position : static | relative | absolute | fixed | sticky
```

- static: caja normal, presentada de acuerdo al flujo normal. (’top’,’right’, ’bottom’, y ’left’ no se aplican).
- relative: la posición de la caja es calculada de acuerdo al flujo normal, la caja es deslizada de modo relativo a su posición normal.
- absolute: la posición (y tamaño) de la caja se especifican con las propiedades ’top’, ’right’, ’bottom’, y ’left’.
- fixed: la posición de la caja es calculada de acuerdo al modelo ’absolute’, pero, además, la caja se fija con respecto a alguna referencia.
- Sticky: Mantiene el elemento en el flujo normal, al alcanzar una posición durante el desplazamiento permanece visible dentro de su contenedor.

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

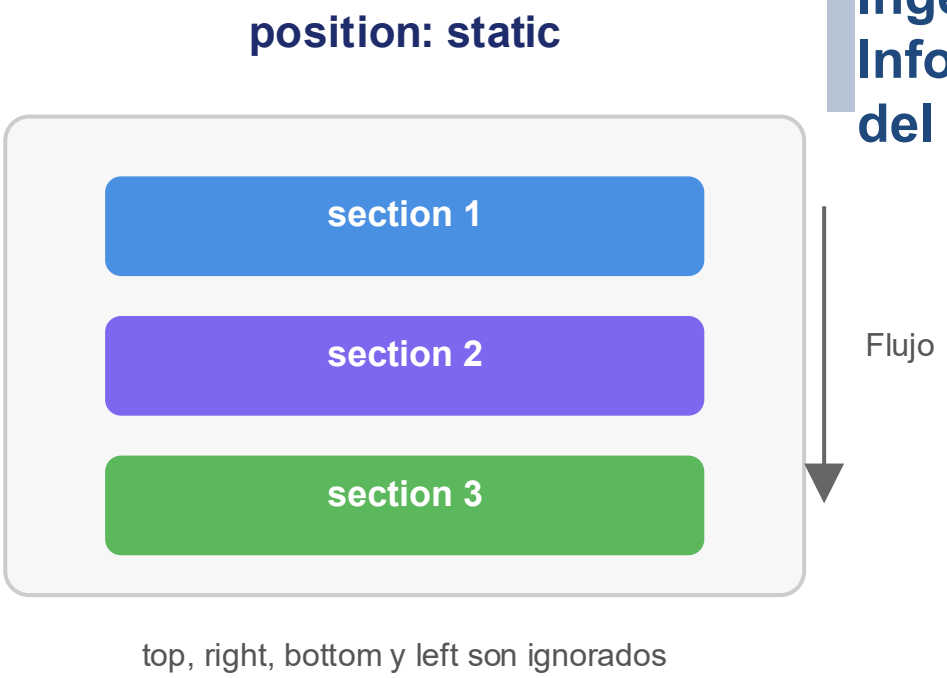

```
position : static
```

Valor por defecto.

```
section {
    position: static;
}
```

Características:

- Sigue el flujo normal.
  - Ignora top, bottom, left y right.
- No cambia su posición.



Posicionamiento tradicional

Posicionamiento relativo

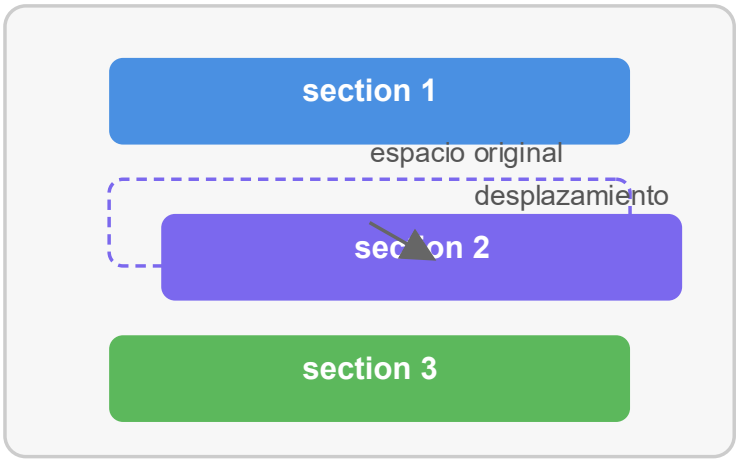
- La caja es situada de acuerdo al flujo normal y luego puede moverse con relación a su posición
- Mantienen su tamaño de flujo normal, incluyendo los saltos de línea y el espacio reservado para ellos.
- Otra posibilidad es marcar un elemento como “relative” sin cambiar realmente su posición, sólo para que sirva como contenedor de otros elementos con posicionamiento absoluto

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional


```
   position : relative
```

```
section {
    position: relative;
    top: 20px;
    left: 30px;
}
```



Características:

El elemento se mueve visualmente, pero mantiene su posición original en el flujo

- Mantiene su espacio original
- Se desplaza visualmente.
- Sirve como referencia para elementos absolutos.

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

Bloque de contención o contenedor

- La posición y tamaño de las cajas de un elemento son computadas en relación a cierto rectángulo, llamado el bloque de contención del elemento.
  - Position: fixed el bloque contenedor es la pantalla
  - Position: relative
    - no existe bloque contenedor, las propiedades top, right, bottom y left mueven el elemento respecto a su posición normal.
    - PERO pueden ser bloque contenedor para posicionar otros elementos.
  - Position: absolute el contenedor suele ser el elemento raiz o si están dentro de otro elemento posicionado será este elemento padre.

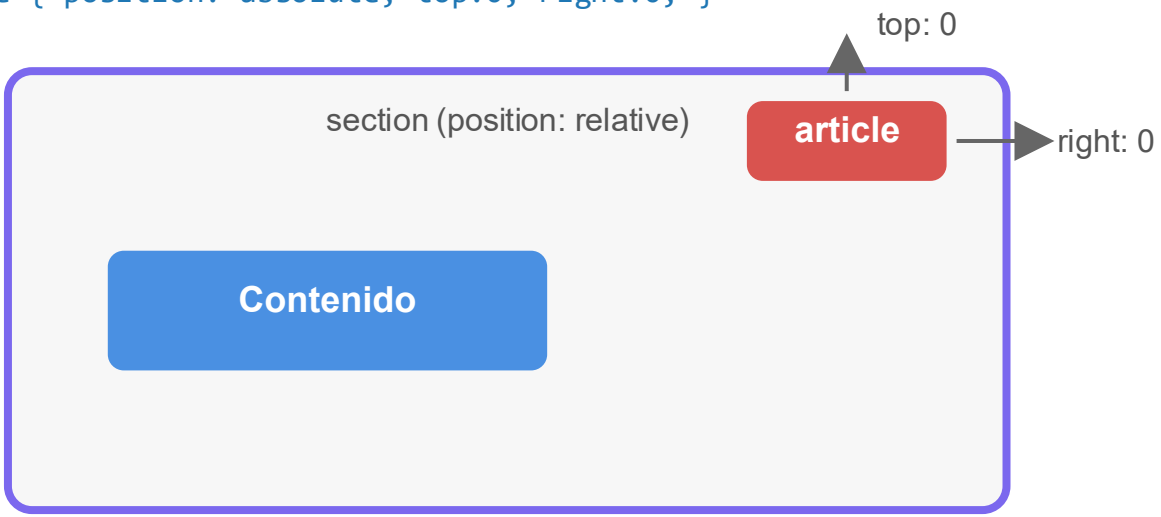
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

Bloque de contención o contenedor

Bloque contenedor para position:absolute

```
section { position: relative; }
article { position: absolute; top:0; right:0; }
```



El article se posiciona respecto a los bordes de la section

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

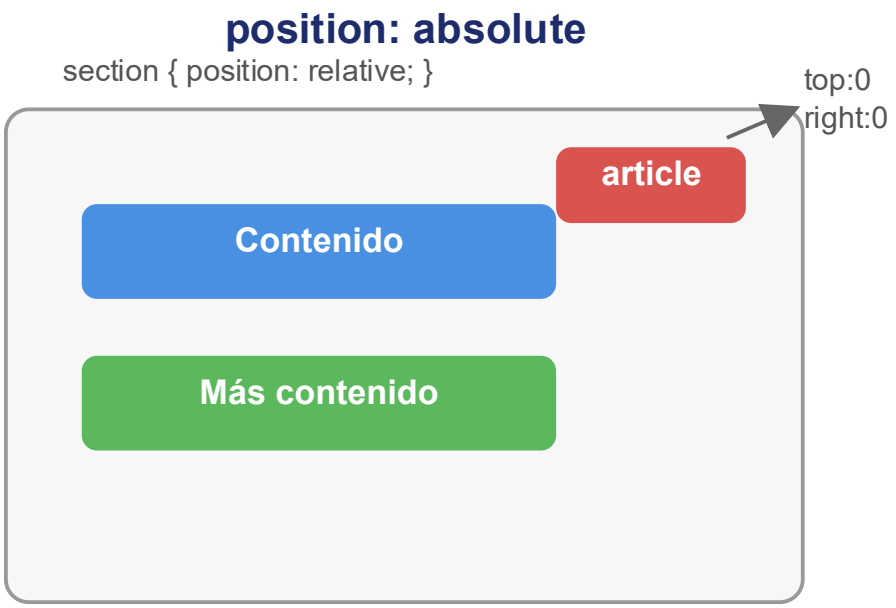
Posicionamiento Absoluto y ("fixed")

- Permite mantener un elemento siempre en la misma posición de la pantalla (p. ej. un logotipo)
- Generalmente en estos casos indicaremos su ancho (width), además de su posición (mediante left o right y top o bottom).También se puede indicar left y right a la vez y no especificar el ancho
- También permite dividir un documento en varias partes
  - Se utiliza para elementos superpuestos o independientes del flujo normal
  - Si están dentro de otro elemento posicionado (ya sea relativo, absoluto o fijo), ése será su elemento contenedor

Posicionamiento tradicional


```
   position : absolute

section {
    position: relative;
}
article {
    position: absolute;
    top: 0;
    right: 0;
}
```



El elemento absoluto sale del flujo normal y se posiciona respecto al primer ancestro con position distinto de static

Características:

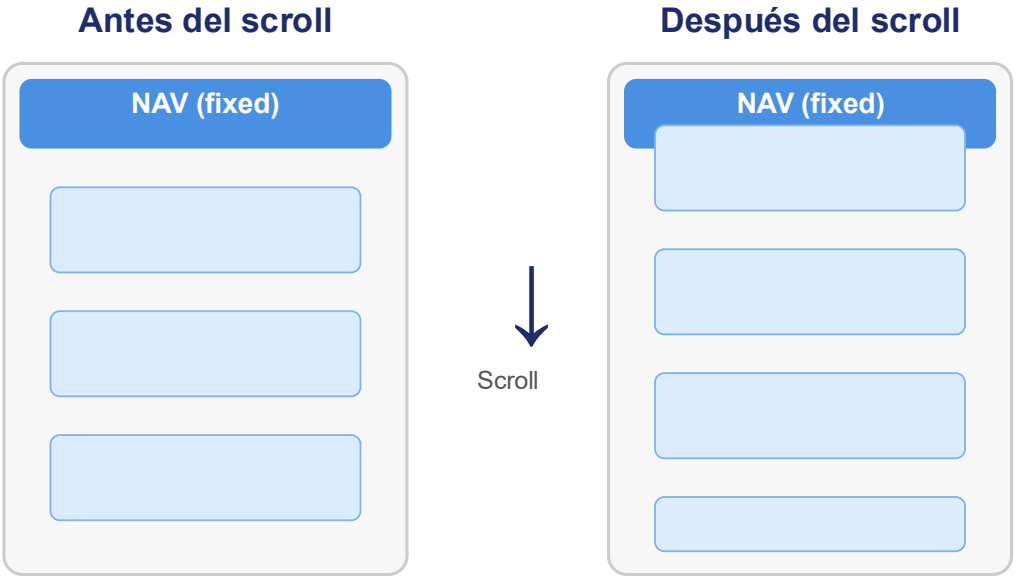
- Sale del flujo normal.
- Puede superponerse a otros elementos.
- Muy usado para iconos y etiquetas.

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional


```
position : fixed
```

```
nav {
    position: fixed;
    top: 0;
    width: 100%;
}
```



El elemento permanece visible porque está fijado a la ventana del navegador

Ejemplos:

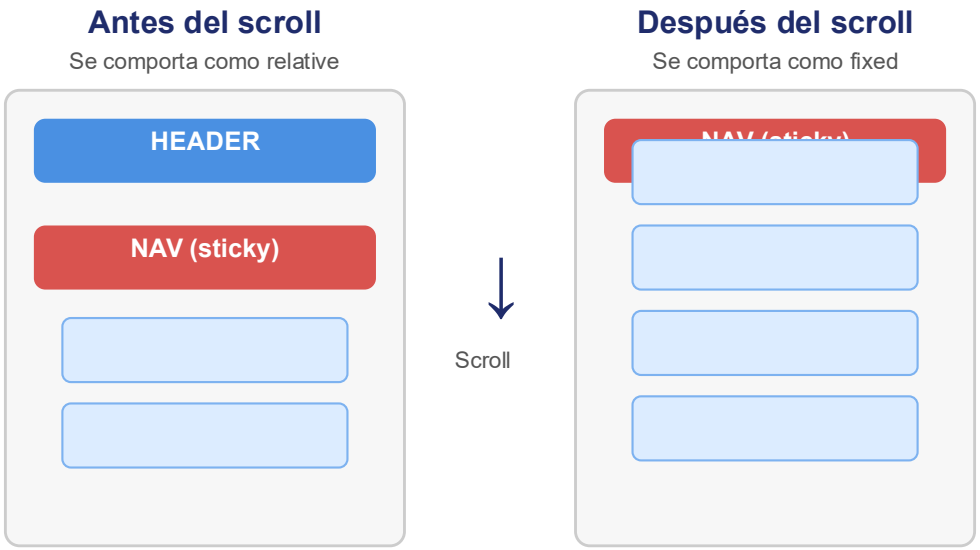
- Menús superiores.
- Botones de ayuda.
- Accesos rápidos.
- El elemento permanece visible durante el scroll.

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional


```
position : sticky
```

```
nav {
    position: sticky;
    top: 0;
}
```



Comportamiento:

- Funciona como relative.
- Al hacer scroll se comporta como fixed
- Muy frecuente en barras de navegación

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

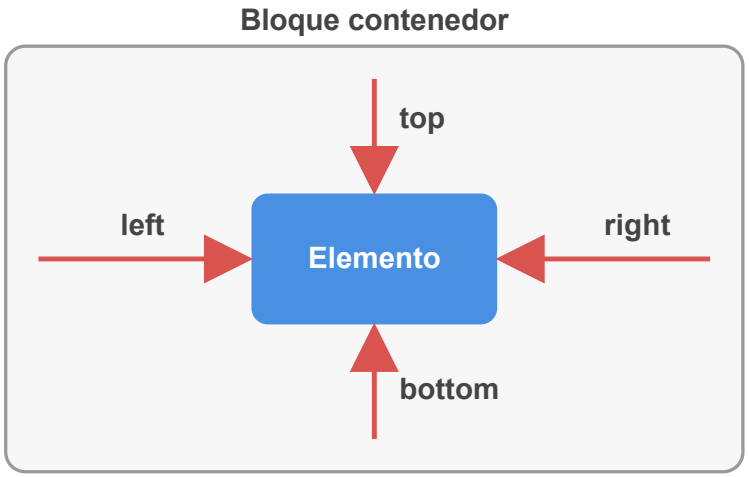
Posicionamiento tradicional

Desplazamiento de las cajas

`top, right, bottom, left : <medida> | porcentaje>|auto`

- Los elementos posicionados (diferentes de static) generan cajas posicionadas, situadas de acuerdo estas cuatro propiedades

top · right · bottom · left



`top:20px; left:30px;`

Indican la distancia respecto a los bordes del bloque contenedor

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

Posicionamiento flotante (histórico)

`float: left | right | none`

- Desplaza una caja a la izquierda o a la derecha
- Permite que el contenido fluya a su alrededor
- Durante años se utilizo para construir layouts

`clear: none | left | right | both`

- Evita elementos flotantes en un determinado lado

No debe utilizarse, Flexbox y Grid son la solución recomendada actualmente

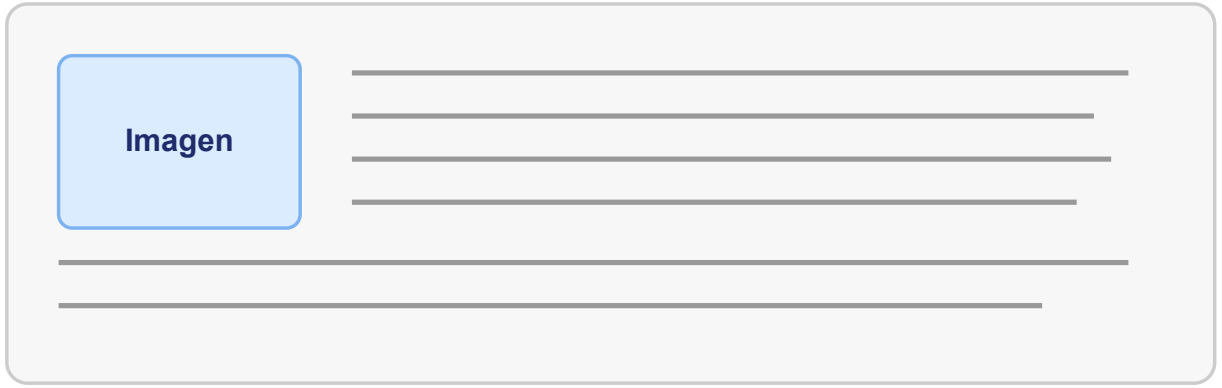
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

Posicionamiento flotante (histórico)

float: left

```
img { float: left; }
```



El contenido fluye alrededor del elemento flotante

No debe utilizarse, Flexbox y Grid son la solución recomendada actualmente

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Posicionamiento tradicional

Problemas del posicionamiento tradicional

- Adaptabilidad del diseño
- Centrar verticalmente los elementos
- Crear columnas equilibradas
- Mantener layouts complejos

Solución actual es:

- Flexible Box Layout (Flexbox)
- CSS Grid Layout
- Media Queries

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grado en Ingeniería Informática del Software

FLEXIBLE BOX LAYOUT

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Flexible Box Layout

Flexbox es un sistema de diseño para organizar elementos en una única dimensión.

- Horizontalmente
- Verticalmente

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Software y estándares para la Web

Flexible Box Layout

HTML

```
<nav>
   <a>Inicio</a>
   <a>Productos</a>
   <a>Contacto</a>
</nav>
```

CSS

```
nav {
   display: flex;
}
```

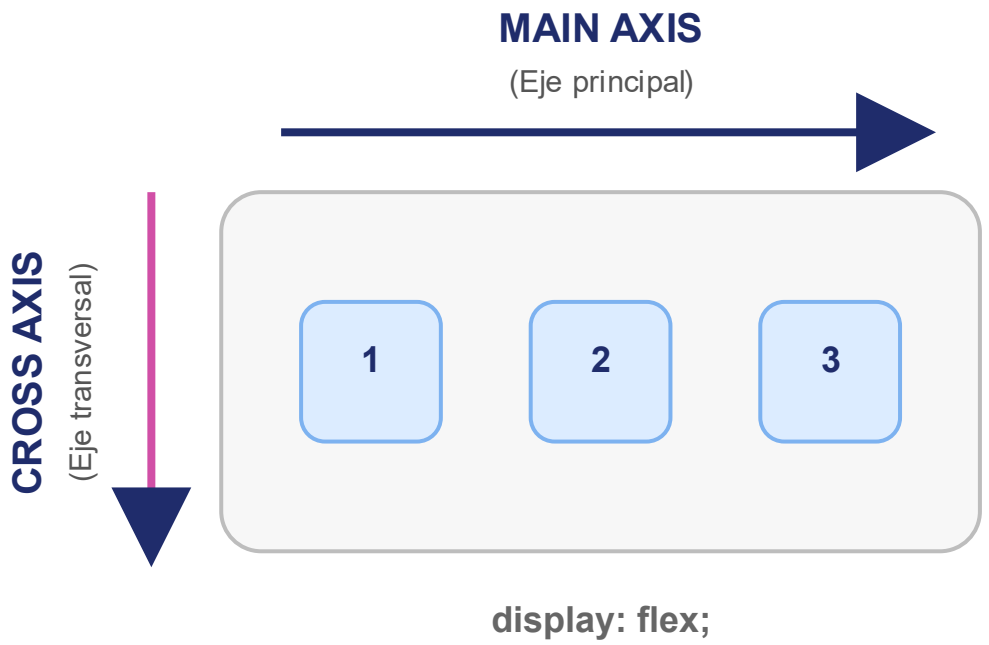
Resultado



nav (contenedor flex)

Ejes de Flexbox

- Todas las propiedades de Flexbox trabajan sobre estos dos ejes.



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

justify-content Alinea sobre el eje principal.

```
nav {
    display: flex;
    justify-content: space-between;
}
```

Opciones frecuentes:

- flex-start
- center
- flex-end
- space-between
- space-around
- space-evenly



Los elementos se distribuyen dejando el mismo espacio entre ellos

Software y estándares para la Web

Flexible Box Layout

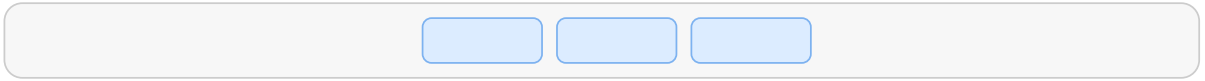
Grado en Ingeniería Informática del Software

justify-content

flex-start: los elementos se colocan al inicio del contenedor



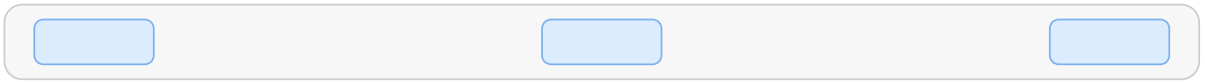
center: los elementos se agrupan en el centro



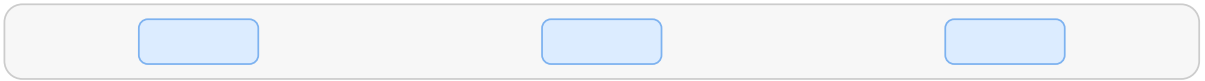
flex-end: los elementos se colocan al final del contenedor



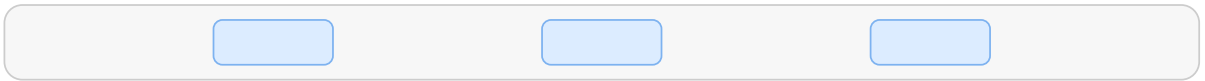
space-between: el primer y ultimo elemento tocan los extremos y el espacio queda entre los elementos



space-around: cada elemento con espacio a ambos lados - el espacio de los extremos es menor



space-evenly: todo el espacio es igual, los elementos y bordes mantienen la misma separación



Escuela de Ingeniería Informática
Universidad de Oviedo

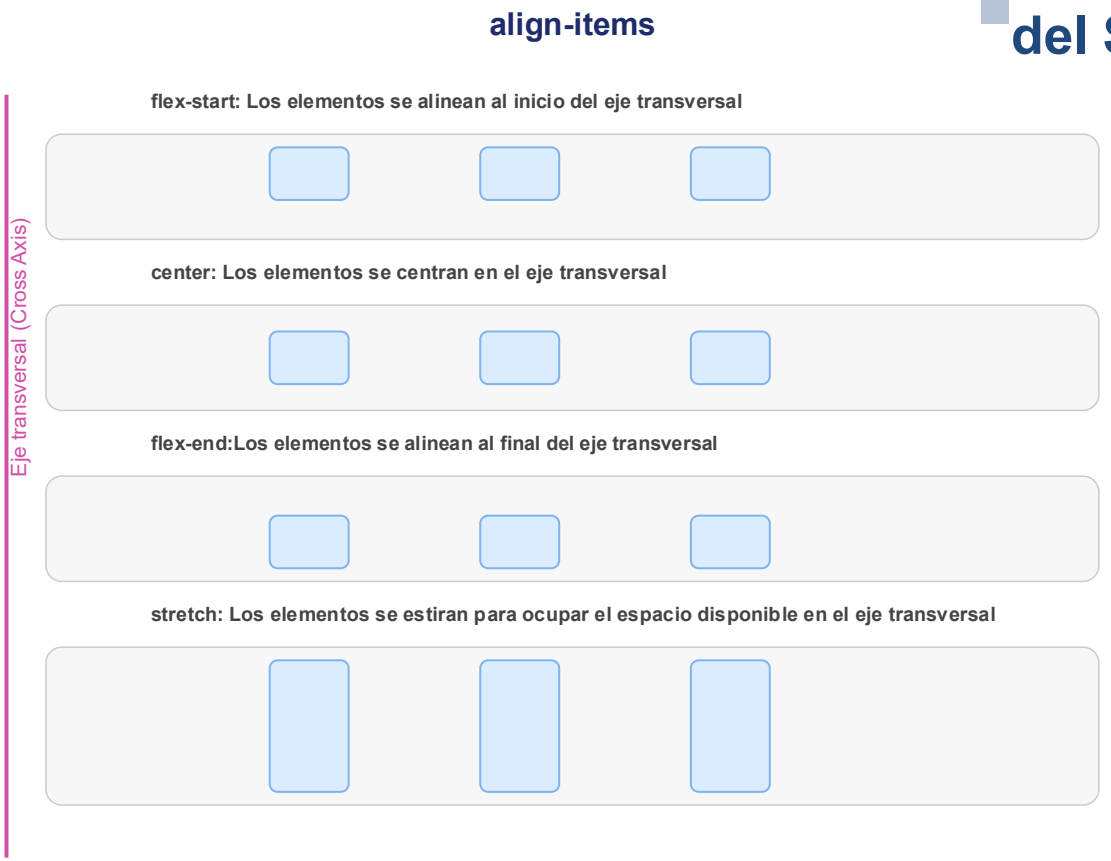
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

align-items Alinea sobre el eje transversal.

```
nav {
    display: flex;
    align-items: center;
}
```

Opciones:

- flex-start
- center
- flex-end
- stretch



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

flex-direction

- Fila (valor por defecto):

```
nav {
  display: flex;
  flex-direction: row;
}
```

- Columna:

```
nav {
  display: flex;
  flex-direction: column;
}
```



flex-wrap

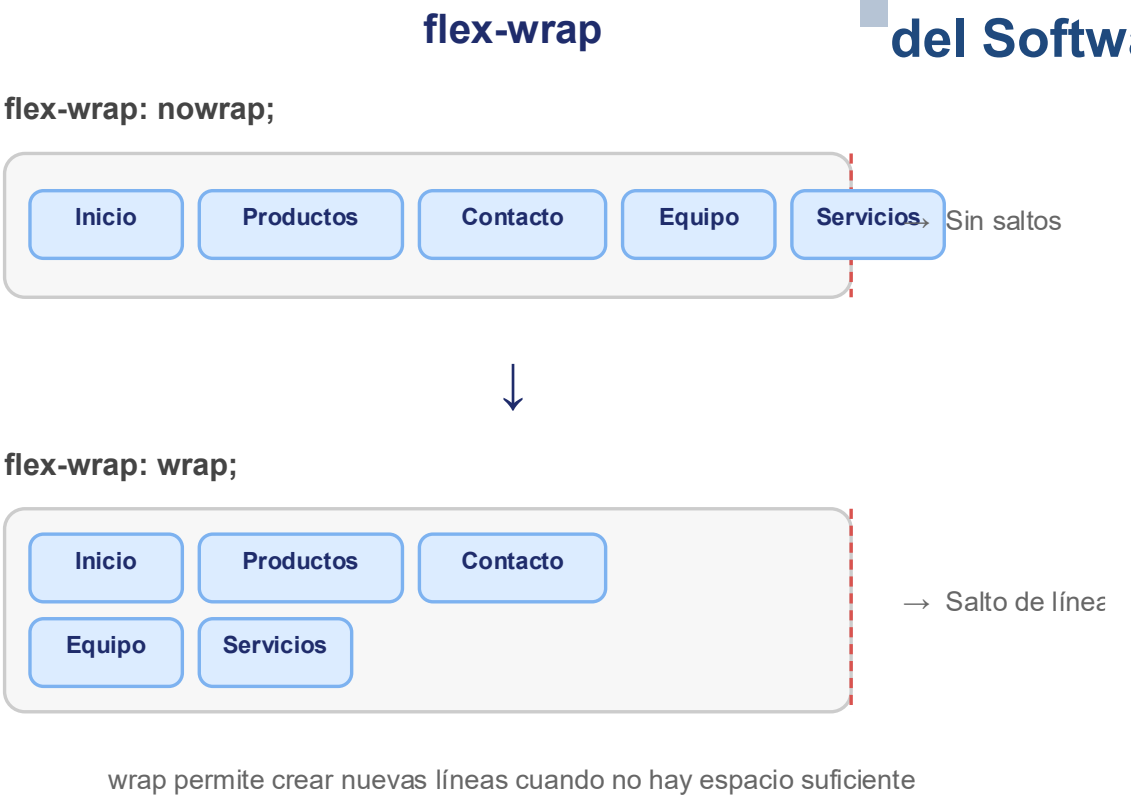
- Permite saltos de línea.

HTML:

```
<nav>
  <a href="">Inicio</a>
  <a href="">Productos</a>
  <a href="">Contacto</a>
</nav>
```

CSS:

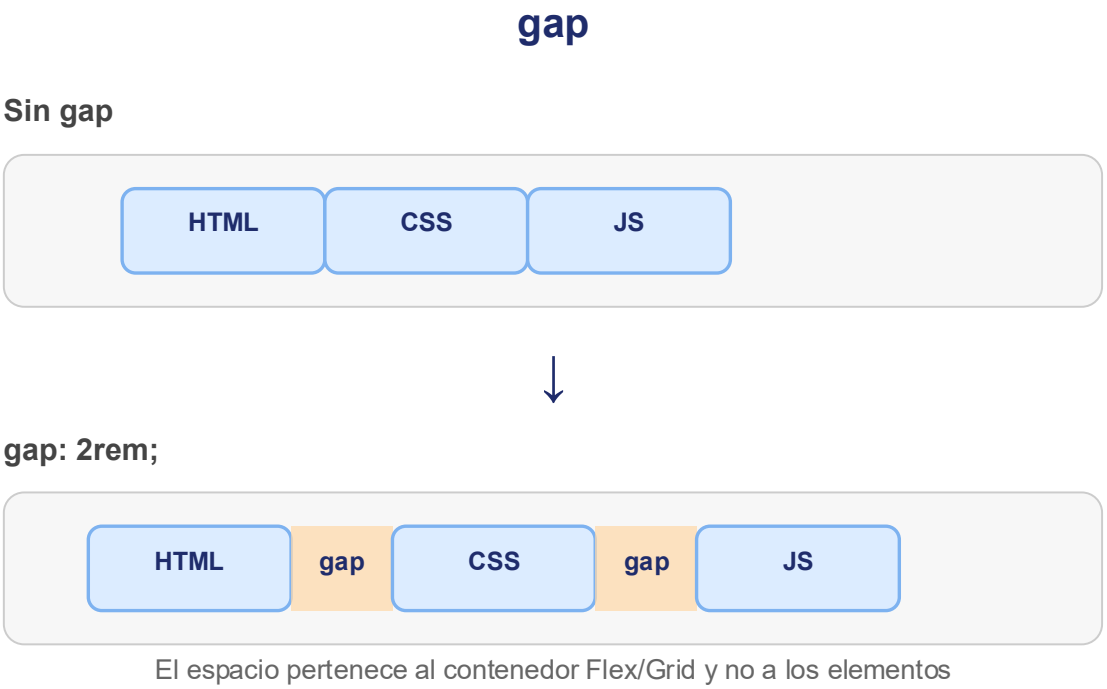
```
nav {
    display: flex;
    flex-wrap: wrap;
}
```



gap

- Separación entre elementos, pertenece al contenedor Flex o Grid (no al elemento)

```
main {
    display: flex;
    gap: 2rem;
}
```



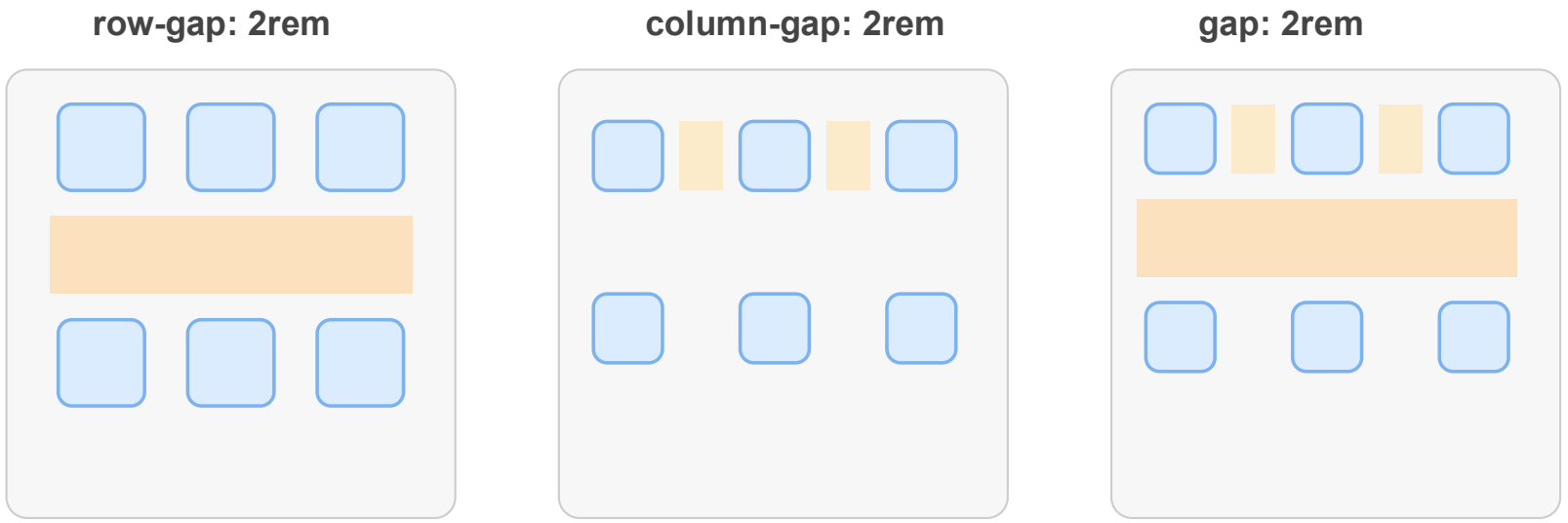
El espacio pertenece al contenedor Flex/Grid y no a los elementos

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

gap es una propiedad abreviada

- row-gap y columna-gap

row-gap, column-gap y gap



row-gap → separación entre filas

column-gap → separación entre columnas

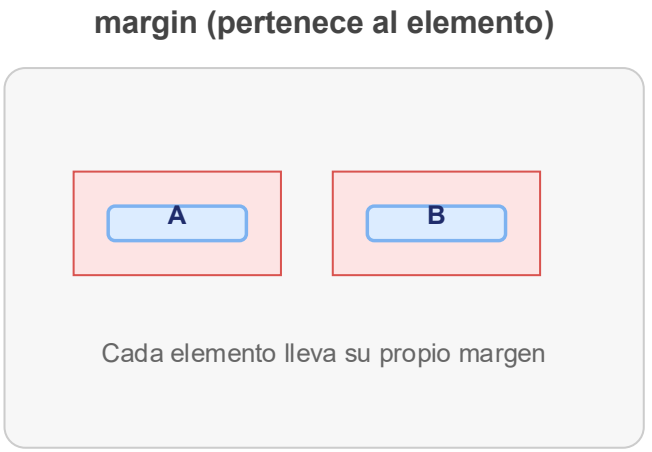
gap → row-gap + column-gap

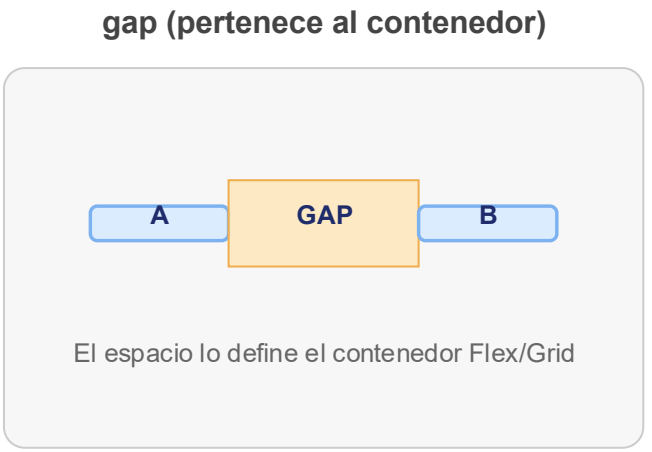
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

gap o margin

```
article {
    margin: 1em;
}

main {
    display: flex;
    gap: 1em;
}
```





- Margin pertenece al elemento.
- Gap pertenece al layout.

¿Cuándo usar Flexbox?

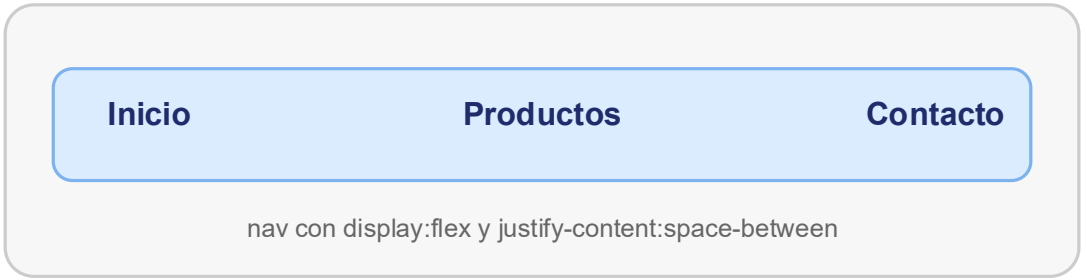
- Menús
- Botones
- Formularios
- Tarjetas
- Barras de herramientas

Regla rápida:

- Una dimensión → Flexbox

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Ejemplo




```
<nav>
  <a href="">Inicio</a>
  <a href="">Productos</a>
  <a href="">Contacto</a>
</nav>
nav {
    display: flex;
    justify-content: space-between;
}
```

Muy parecido a una barra de navegación real.

Grado en Ingeniería Informática del Software

CSS GRID LAYOUT

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grid permite trabajar con:

- Filas y Columnas al mismo tiempo
- Sistema bidimensional

```
main {
    display: grid;
}
```

- A partir de aquí definimos la estructura de la cuadrícula.

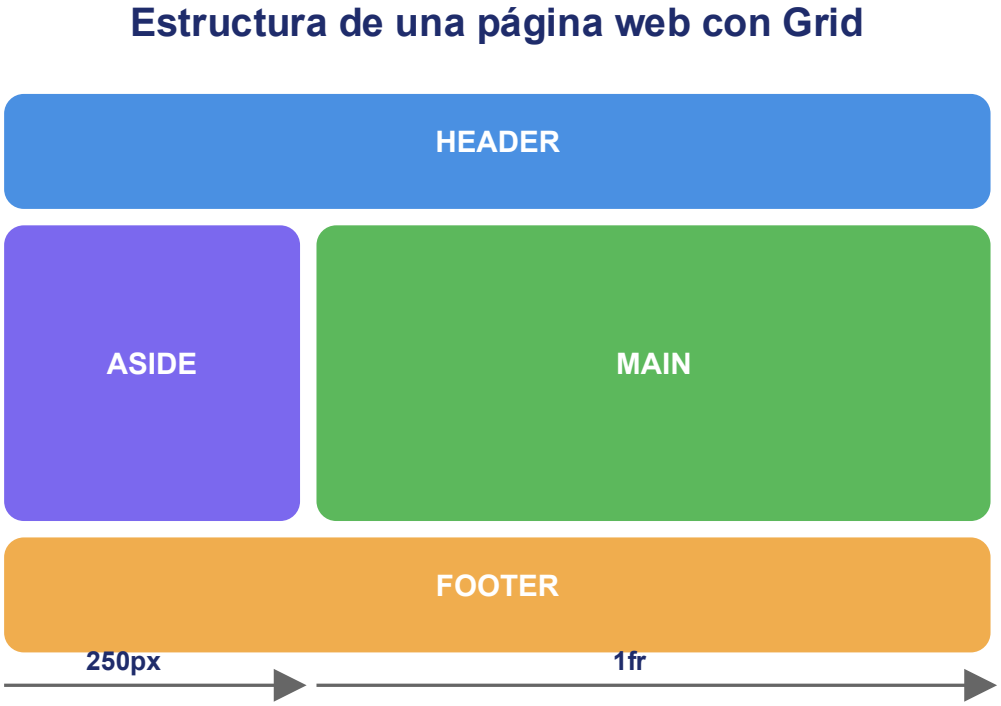
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

CSS Grid Layout

- Grid para una página web

HTML:

```
<header></header>
<aside></aside>
<main></main>
<footer></footer>
```



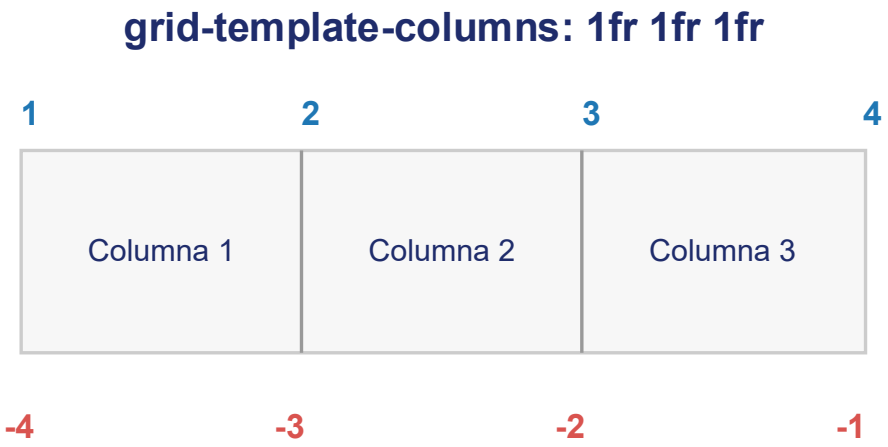
CSS:

```
body {
    display: grid;
    grid-template-columns: 250px 1fr;
}
```


Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Definir columnas

```
main {
   display: grid;
   grid-template-columns: 1fr 1fr 1fr; /*3 columnas iguales*/
}
```



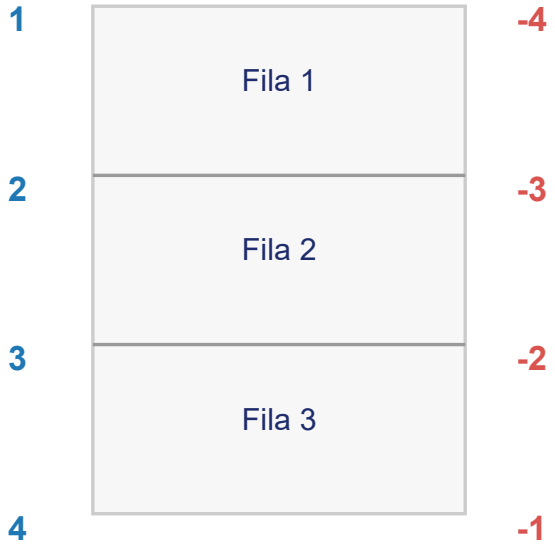
Las líneas pueden referenciarse desde el inicio o desde el final de la cuadrícula

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Definir filas

```
main {
    display: grid;
    grid-template-rows: 1fr 1fr 1fr; /*3 filas iguales*/
}
```

grid-template-rows: 1fr 1fr 1fr

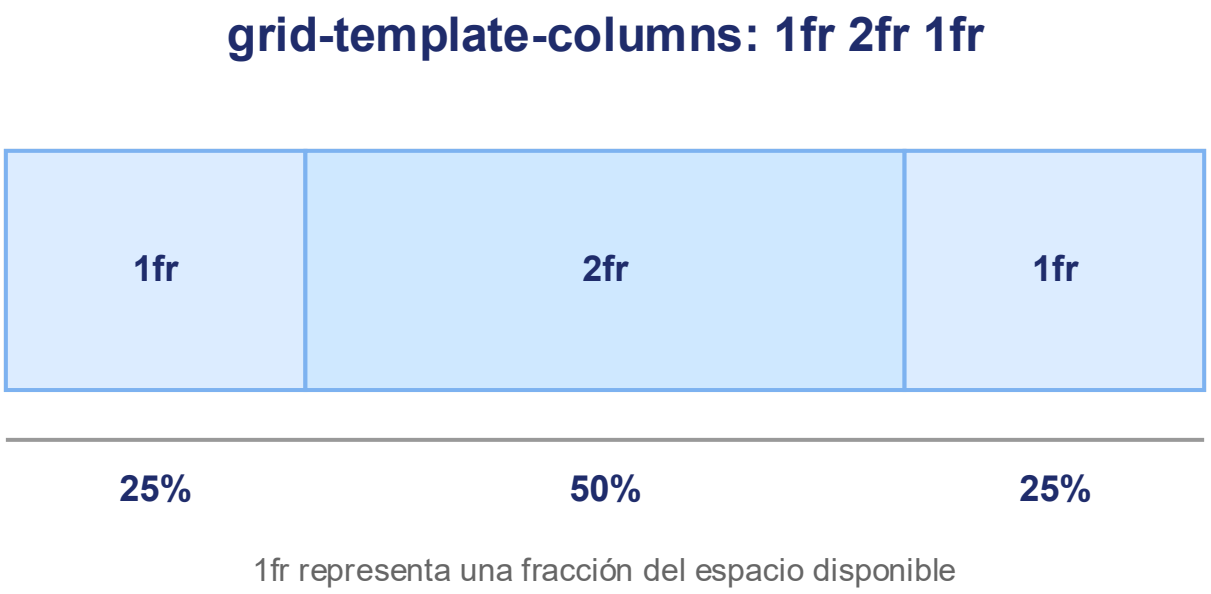


Las líneas de fila pueden referenciarse desde el inicio o desde el final

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

La unidad fr

- grid-template-columns: 1fr 2fr 1fr;
  - Representa una fracción del espacio disponible.
  - 25% 50% 25% aproximadamente.



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

La notación repeat()

- Evita repetir el mismo patrón

grid-template-columns: repeat(3, 1fr)

```
repeat(3, 1fr) = 1fr 1fr 1fr
```



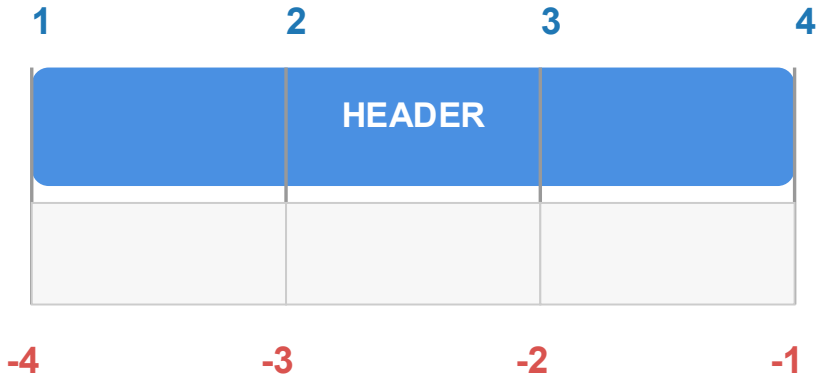
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

grid-column

- Propiedad abreviada grid-column-start y grid-column-end

```
header{
grid-column: 1 / -1;
        /*grid-column-start: 1 y grid-column-end:-1 */
}
```

Numeración de líneas en CSS Grid



header { grid-column: 1 / -1; }

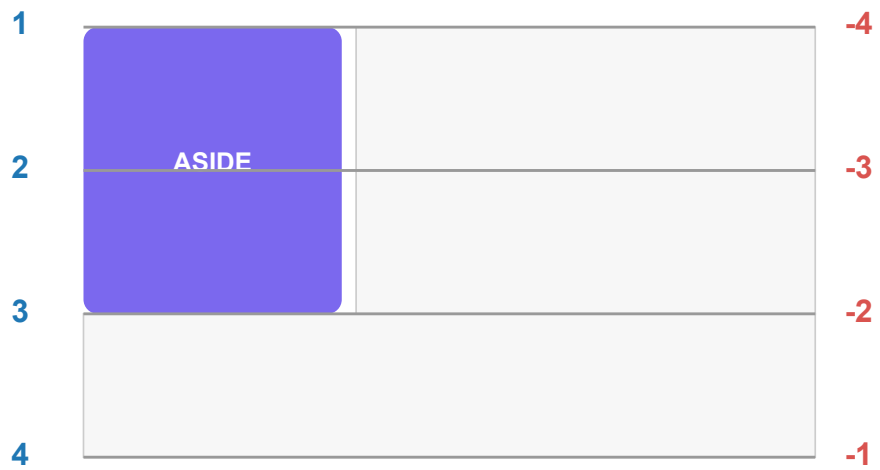
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

grid-row

- Propiedad abreviada grid-row-start y grid-row-end

```
aside{
grid-row: 1/3; /*grid-row-start: 1 y grid-row-end:3 */
}
```

Numeración de filas en CSS Grid



aside { grid-row: 1 / 3; }

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grid Areas

- Permite asignar nombres a regiones de una cuadrícula para definir la estructura de una página de forma visual

```
body {
    display: grid;
    grid-template-areas:
        " cabecera cabecera"
        " adicional principal"
        " pie pie";
}
```

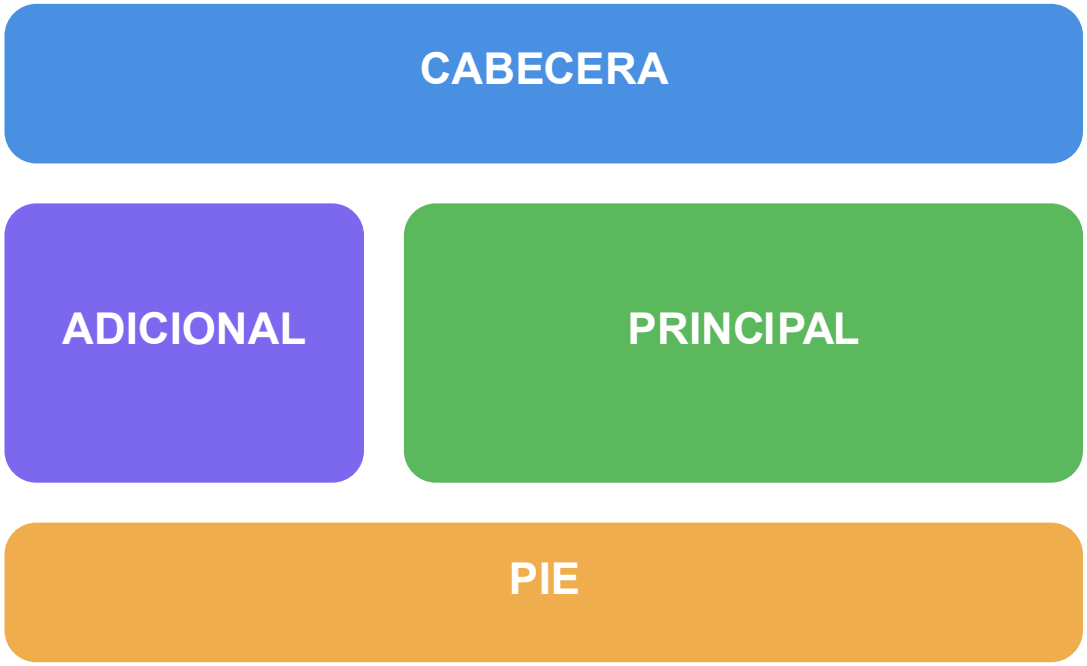
- Cada cadena representa una fila de la cuadrícula y cada palabra una zona de la página
- Las palabras repetidas indican que un área ocupa varias celdas

```
header{ grid-area: cabecera;}
main{grid-area : principal;}
aside{grid-area : adicional;}
footer{ grid-area: pie;}
```


Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grid Areas

grid-template-areas



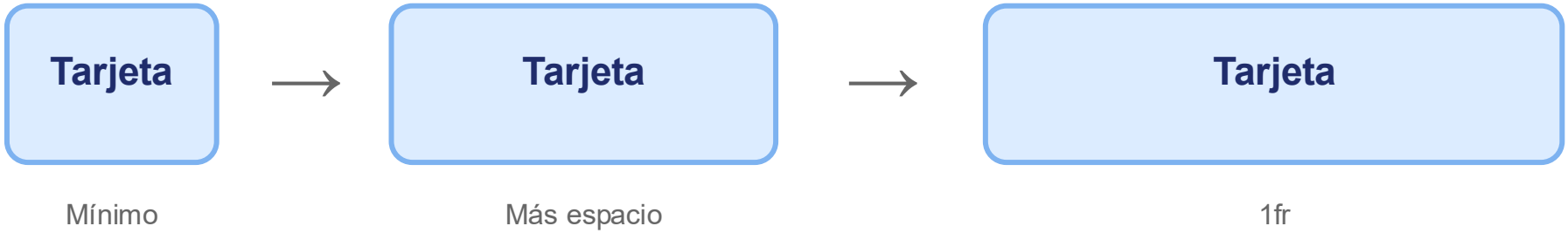
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Función minimax()

- Permite definir un tamaño mínimo y un tamaño máximo para una fila o columna
- Define el rango de tamaños para una fila o columna

```
minmax(200px, 1fr)
```

- Permite crear diseños adaptables



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

auto-fill

- Crea tantas columnas como sea posible.
  - Mantiene las columnas aunque estén vacías
- Reserva el espacio para futuras columnas

```
grid-template-columns:repeat(auto-fill, minmax(200px,1fr));
```



Cuando utilizarlo:

- Reservar espacio para futuras columnas
- Diseños con estructura fija
- Casos donde interesa mantener el número de columnas.

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

auto-fit

- Crea tantas columnas como sea posible
  - Las columnas vacías desaparecen
- El espacio libre se redistribuye.

```
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
```



Cuando utilizarlo:

- Aprovechar todo el espacio disponible
- Interfaces responsive
- Diseño basado en tarjetas

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

auto-fit o autofill

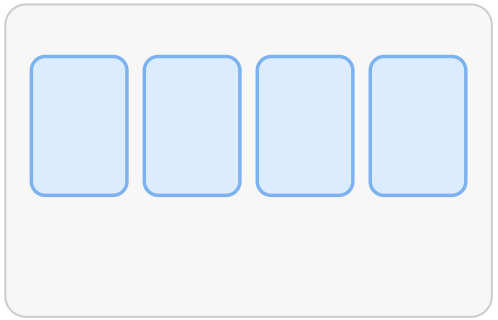


Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Diseño responsive con auto-fit

repeat(auto-fit, minmax(250px,1fr))

Escritorio



4 columnas

Tablet



2 columnas

Móvil



1 columna

El número de columnas se adapta automáticamente manteniendo un ancho mínimo de 250px

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Propiedad abreviada grid-template

- Permite definir filas, columnas y áreas en una sola instrucción.
- Agrupa:
  - grid-template-rows
  - grid-template-columns
  - grid-template-áreas

```
grid-template:
 "header header" 100px
 "aside  main"   1fr
 "footer footer" 80px
          / 250px 1fr;
```

Equivale a:

```
grid-template-areas:
    "header header"
    "aside main"
    "footer footer";
grid-template-rows:
    100px 1fr 80px;
grid-template-columns: 250px 1fr;
```


Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Propiedad abreviada grid

- grid-template
- grid-template-rows
- grid-template-columns
- grid-template-areas

```
grid:
 "header header" 100px
 "aside  main"   1fr
 "footer footer" 80px
          / 250px 1fr;
```

Equivale a:

```
grid-template-areas:
    "header header"
    "aside main"
    "footer footer";
grid-template-rows:
    100px 1fr 80px;
grid-template-columns: 250px 1fr;
```


Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grado en Ingeniería Informática del Software

FLEXIBLE BOX LAYOUT VERSUS CSS GRID LAYOUT

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Flexible box layout versus CSS Grid layout

Flexbox

- Una dimensión
- Fila o columna
- Para componentes

Grid

- Dos dimensiones
- Filas y columnas
- Para la estructura

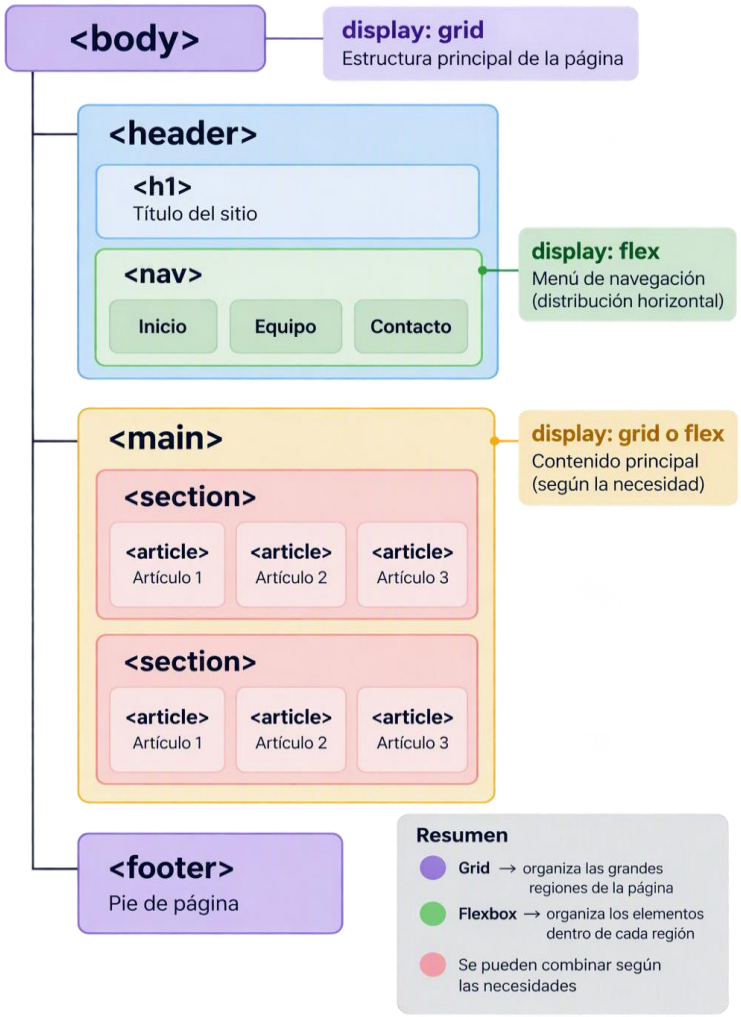
Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Software y estándares para la Web

Flexible box layout versus CSS Grid layout

Ejemplo típico

- Grid para la estructura general
  - organiza las grandes regiones del documento
- Flexbox para organizar el contenido interno
  - organiza los elementos dentro de cada región
- Se pueden combinar



Escuela de Ingeniería Informática
Universidad de Oviedo

Grado en Ingeniería Informática del Software

MEDIA QUERIES

Escuela de Ingeniería Informática
Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Media Queries

- Sintaxis:

```
@media (max-width: 768px) {
}
```

- Permiten aplicar estilos según determinadas condiciones.

- Breakpoints
  - Ancho de la pantalla a partir del cual se aplican nuevos estilos
  - 768px para móviles
  - 1024px para tabletas

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Flex Responsive

Escritorio

```
nav {
    display: flex;
    flex-direction: row;
}
```

Móvil

```
@media (max-width: 768px) {
    nav {
     flex-direction: column;
    }
}
```



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Escuela de Ingeniería Informática
Universidad de Oviedo

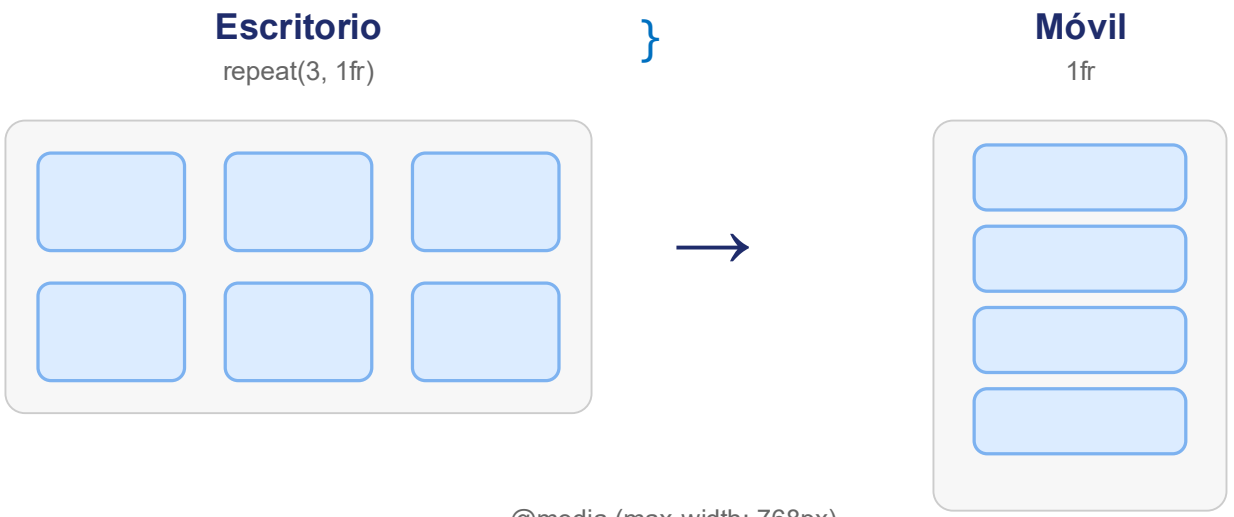
Grid Responsive

Escritorio

```
main {
 display: grid;
 grid-template-columns:
                repeat(3,1fr);
}
```

Móvil

```
@media (max-width: 768px){
 main {
  grid-template-columns: 1fr;
 }
}
```



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Media Queries

Mobile first

Primero para móvil

```
main {
 display: grid;
 grid-template-columns: 1fr;
}
```

Se adapta para pantalla

```
@media (min-width: 768px) {
 main {
  grid-template-columns: repeat(3, 1fr);
    }
}
```

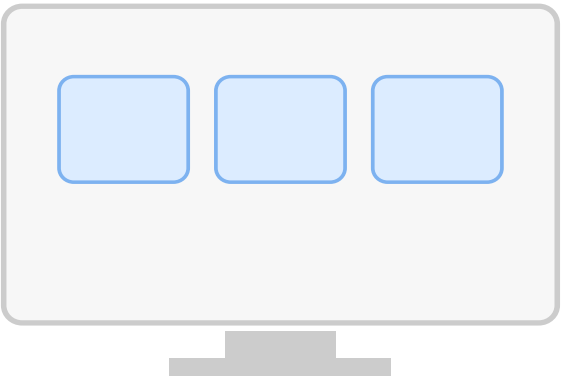
1. Diseño móvil

Sin media queries



2. Pantallas grandes

@media (min-width:768px)



Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grado en Ingeniería Informática del Software

CONCLUSIONES

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Conclusiones

Regla de oro

- HTML semántico para estructurar el contenido
- Flexbox para organizar componentes en una dimensión
- Grid para construir la estructura general de la página
- Combinar Flexbox y Grid
- Media Queries para adaptar la interfaz a distintos tamaños de pantalla

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Grado en Ingeniería Informática del Software

REFERENCIAS

Escuela de Ingeniería Informática Universidad de Oviedo

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)

Referencias

- Página de referencia en el W3C para hojas de estilo https://www.w3.org/Style/CSS/
- Lista de todos los módulos y su estado actual https://www.w3.org/Style/CSS/current-work
  - CSS Positioned Layout
  - CSS Flexible Box Layout
  - CSS Grid Layout Module
  - Media Queries

Dra. B. Cristina Pelayo García-Bustelo (crispelayo@uniovi.es)